

Mass Trait Reference

Every Mass Entity trait, what it does, and the one rule you must not break.

How to build a Mass crowd/agent with LCM Nav3D. Add these traits to a **Mass Entity Config** asset ([Create](#) → [Miscellaneous](#) → [Mass Entity Config](#)) and use it on an [AMassSpawner](#) .

NOTE **One rule:** every agent needs **exactly one movement trait**. Everything else is an orthogonal add-on. The editor validator (below) enforces this.

The traits

 LCM Nav3D Agent : the one movement trait (start here)

The single authoring entry point. Pick a **Navigation Mode**:

Navigation Mode	What it does	Fragments composed
Path Following	Per-agent A* / GPU path through the SVO (HPA* across chunks). Best for hero/mid-count agents, precise routes, infinite worlds.	NavAgent , PathRequest , PathResult
Flow Field	One shared Dijkstra flow field per goal, O(1)/agent sampling. Best for 10K-scale crowds sharing goals (finite worlds).	FlowField (state) + FlowConfigShared (const shared config)

Mode-specific properties hide automatically via [EditCondition](#) . Common movement knobs (radius, speed, accel, turn-rate, avoidance style, flight style, lane bias) apply to both.

 LCM Nav3D Goal : auto-pathing (Path Following only)

Points a **Path Following** agent at a fixed location or a tagged actor and keeps it pathed there (re-pathing on drift, optional lead-prediction for moving targets). **Not for Flow Field**. Set the goal on the Agent trait's *Goal Mode* properties instead. The validator warns if you get this wrong.

+ LCM Nav3D Perception : SVO-LOS sight (either mode)

Makes an agent both a *senser* and *perceivable* (faction-based, true-3D SVO line-of-sight). Writes a per-agent "closest visible threat" surface read by EQS, StateTree, or steering. Opt-in avoidance:

`r.LcmNav.Mass.PerceptionAvoid 1` makes Flow Field agents veer from their own perceived threat.

🎨 Rendering: pick ONE

Trait	Use when	LOD?	Animation
LCM Nav3D ISM Render	Turnkey, hundreds to low thousands, "just draw a mesh".	No	Yes, 2 per-instance custom-data floats: <code>[0]</code> speed01 (walk/run blend), <code>[1]</code> stable phase (de-sync). Feed a VAT/WPO material.
LCM Nav3D Representation	LOD-culled crowds; optional hero actor up close. Wraps Epic MassRepresentation with sane defaults (draws ISM at all visible LODs, culls far). Needs a viewer/player in the level for LOD distance.	Yes	Via the mesh material / actor
(Niagara GPU crowd)	10K+ (deferred / roadmap)	-	VAT

Combination matrix

	Agent (Path)	Agent (Flow)	Goal	Perception	ISM Render	Representation
Agent (Path)	-	✗ conflict	✓	✓	✓	✓
Agent (Flow)	✗ conflict	-	⚠ use Agent goal	✓	✓	✓
Goal	✓	⚠ redundant	-	✓	✓	✓
Perception	✓	✓	✓	-	✓	✓
ISM Render	✓	✓	✓	✓	-	✗ pick one
Representation	✓	✓	✓	✓	✗ pick one	-

- ✗ **conflict**: two movement stacks; the editor validator errors and (at runtime) the two steerers mutually exclude → the entity would not move. Keep one.
- ⚠ = allowed but pointless; the validator explains.

The editor validator

`ULcmMassConfigValidator` runs on every `UMassEntityConfigAsset` (right-click → Validate, or on save/cook). It errors on:

1. **Two movement stacks** (Path + Flow, in any mix of unified/legacy traits).
2. **Goal without a Path Following agent**.

So a broken config can't ship. Requires the **Data Validation** plugin (bundled).

Migration from the old traits

The pre-2026-07 traits still work (byte-identical archetypes) but are marked (**Legacy**) and superseded by the unified trait:

Legacy trait	Replace with
<code>LCM Nav3D Agent (Legacy)</code>	<code>LCM Nav3D Agent</code> , NavMode = Path Following
<code>LCM Nav3D Flow Steer (Legacy)</code>	<code>LCM Nav3D Agent</code> , NavMode = Flow Field

Existing content (e.g. `DA_MassAgent_Apex`) keeps working, so there is no forced migration.

Default-value deltas when migrating (the unified trait has ONE default per property; the two legacy traits differed): `ArriveRadius` is now **100** (legacy path 80 / legacy flow 150) and `AvoidanceStyle` defaults to **Reactive** (the legacy Flow Steer defaulted to Predictive/ORCA). If your config relied on a legacy default, set the value explicitly after switching traits. `TargetActorTag` defaults to `ApexGoal` (matches legacy).

World Partition / large-world (City Sample scale): REQUIRED READING

The vanishing-crowd trap (stock engine behavior)

`AMassSpawner::BeginDestroy()` calls `DoDespawning()` : the spawner destroys every entity it spawned when the spawner actor is destroyed (UE 5.8 `MassSpawner.cpp:186`). In a World Partition map a placed spawner is **spatially loaded by default**, so when the player leaves the spawner's cell, the spawner streams out and **silently deletes the entire crowd**. Symptom: agents work near the spawn area, then all disappear when the player walks away.

Fix (pick one):

1. Use `LCM Nav3D Crowd Spawner` instead of the stock Mass Spawner. It is identical (EQS generators etc. all inherited) but defaults to *not* spatially loaded, so its crowd's lifetime is never tied to the player's

position.

2. On a stock `AMassSpawner`: Details → World Partition → Is Spatially Loaded = false.

This follows the AAA rule: **population lifetime must be owned by something persistent** (an always-loaded actor/subsystem), never by a streamed actor. Distance-based *despawning* should be an explicit population policy (an ambient ring, à la City Sample's crowd), not a streaming side effect.

City-scale checklist

- **Navigation:** a city does not fit the finite default (`WorldExtent` = 100 m). Use the manager's **Infinite World** mode (Hierarchical HPA*); the flow crowd keeps its own nav chunks resident via crowd-driven streaming (`r.LcmNav.Mass.CrowdStreaming` , default on), so agents don't lose nav when the player (the only invoker) leaves.
- **Rendering:** `LCM Nav3D Representation` culls agents *visually* past its LOD distances (defaults: Off at 200-250 m, deliberately conservative). For city vistas, raise `LODParams.BaseLODDistance/VisibleLODDistance` on the trait. This is visual LOD only. Entities keep simulating; it is NOT the vanishing-crowd trap above.
- **Ambient population ring** (constant-density crowds around the player with bounded cost, the City Sample pattern) is a roadmap feature. Until then, crowds are persistent: they exist wherever you spawned them.

Useful CVars

CVar	Default	Effect
<code>r.LcmNav.Mass.UseGPU</code>	-1 (per-agent)	Force CPU (0) / GPU (1) pathfinding for Path Following.
<code>r.LcmNav.Mass.PerceptionAvoid</code>	0	1 = Flow agents veer from their perceived closest threat.
<code>r.LcmNav.Mass.PerceptionAvoidRadiusCm</code> / <code>...Strength</code>	600 / 1.0	Perception-avoidance range + strength.
<code>r.LcmNav.Mass.ISMAnimRefSpeedCmS</code>	600	Speed (cm/s) mapping to ISM custom-data <code>speed01 = 1</code> .
<code>r.LcmNav.Mass.Anticipatory</code>	-1 (per-trait)	Route Flow crowds around where obstacles are <i>heading</i> .

Chapter 16 of the LCM Nav3D User Manual.

The complete manual is `LCM_Nav3D_Manual.pdf`, in this folder alongside every other document in the set. Start from `README.pdf`.

LCM Nav3D: True Volumetric 3D Navigation for Unreal Engine. Version 2.05 · Unreal Engine 5.8.

Copyright © 2026 L.Charitha Madhushanka. All Rights Reserved. Proprietary and confidential. Licensed, not sold; use is governed by the licence that accompanies this software. Unreal Engine is a trademark of Epic Games, Inc. All trademarks are the property of their respective owners.